

Final DevSecOps Report

Zero-Trust Vault Semester Project

CI/CD Pipelines, GitLab Configuration, Security Scans, Docker,
Kubernetes, and Automation

Project repository: `devsecops-project`

Main pipeline file: `.gitlab-ci.yml`

Deployment model: Docker Compose and Kubernetes

Academic Year 2025–2026

Contents

Executive Summary	1
1 Project Presentation	2
1.1 Project Context	2
1.2 Project Objectives	2
1.3 Repository Organization	3
2 Application Analysis	4
2.1 Backend	4
2.2 Frontend	4
2.3 SecOps Bot	4
2.4 Monitoring Stack	5
3 DevSecOps Architecture	6
3.1 Architecture Overview	6
3.2 Logical Flow	7
3.3 Security Philosophy	7
4 CI/CD Pipeline Design	8
4.1 Global Pipeline	8
4.2 Pipeline 1: CI Quality and Unit Tests	8
4.2.1 Purpose	8
4.2.2 Stages	9
4.2.3 Expected Evidence	9
4.3 Pipeline 2: SAST and Code Quality	9
4.3.1 Purpose	9
4.3.2 Tools	9
4.3.3 Important Findings	9
4.4 Pipeline 3: Docker Build and Container Scan	10
4.4.1 Purpose	10
4.4.2 Images Built	10

4.4.3	Container Scan	10
4.5	Pipeline 4: DAST and Runtime Monitoring	10
4.5.1	Purpose	10
4.5.2	DAST	10
4.5.3	Runtime Monitoring	11
4.6	Pipeline 5: Deployment and Automation	11
4.6.1	Purpose	11
4.6.2	Deployment Options	11
4.6.3	Automation	11
5	GitLab CI/CD Configuration	12
5.1	Important Variables	12
5.2	Artifacts	12
5.3	Allowed Failures	13
6	DevSecOps Tools Used	14
7	Security Scan Analysis	15
7.1	SAST Analysis	15
7.1.1	Hardcoded JWT Secret	15
7.1.2	Permissive CORS	15
7.1.3	Demo Credentials	15
7.1.4	Hardcoded Frontend API URL	16
7.2	Container Scan Analysis	16
7.2.1	Expected Remediation Process	16
7.3	DAST Analysis	16
7.3.1	Typical Findings	16
7.3.2	Recommended Hardening	17
7.4	Runtime Monitoring Analysis	17
7.4.1	Observed Event Types	17
7.4.2	Operational Value	17
8	Docker Deployment	18
8.1	Docker Compose Services	18
8.2	Execution	18
8.3	Security Notes	18
9	Kubernetes Deployment	19
9.1	Manifests	19
9.2	Deployment Command	19
9.3	Kubernetes Security Improvements	19

10 Automation Scripts	21
10.1 Script Inventory	21
10.2 Automation Benefits	21
11 Risk Register	22
12 Recommendations	23
12.1 Short-Term Improvements	23
12.2 Medium-Term Improvements	23
12.3 Long-Term Improvements	23
13 Conclusion	25
A Useful Commands	26
B Compilation	27

List of Figures

List of Tables

3.1	Architecture components	6
4.1	Pipeline 1 stages	9
6.1	Tools used in the project	14
8.1	Docker Compose services	18
10.1	Automation scripts	21
11.1	Project risk register	22

Executive Summary

This report presents a complete DevSecOps semester project built around a Zero-Trust Vault application. The objective is to demonstrate how security can be integrated continuously into the software delivery lifecycle instead of being executed only at the end of development.

The project contains five pipeline folders, one global GitLab CI/CD configuration, automation scripts, Docker files, Kubernetes manifests, screenshot evidence placeholders, and this final report. The application itself is composed of a FastAPI backend, a React frontend, an ELK monitoring stack, and a SecOps automation bot that can interact with GitLab, Elasticsearch, Telegram, and an LLM provider.

The implemented DevSecOps workflow covers source code quality, automated tests, static analysis, container image building, vulnerability scanning, dynamic application security testing, deployment, runtime log collection, and incident support automation. The report also documents important security findings identified in the source code, including hardcoded secrets, permissive CORS, demo credentials, and hardcoded frontend configuration.

1 Project Presentation

1.1 Project Context

Modern software delivery requires fast releases, repeatable deployments, and continuous security validation. Traditional development models often detect security issues late, when the cost of remediation is higher. DevSecOps addresses this problem by embedding security controls directly into CI/CD pipelines, infrastructure automation, and operational monitoring.

This project applies those principles to a small but realistic web application. The application is intentionally simple enough to understand quickly, but it contains the core elements required to demonstrate a DevSecOps chain:

- A backend API that exposes authentication and protected resources.
- A frontend interface that consumes the backend API.
- Containerized services using Docker.
- GitLab CI/CD jobs for validation and security scanning.
- Docker Compose deployment for local or staging use.
- Kubernetes deployment manifests for cluster delivery.
- Runtime observability using Filebeat, Elasticsearch, and Kibana.
- A SecOps bot that assists with SIEM and pipeline analysis.

1.2 Project Objectives

The main objectives of the project are:

1. Build a complete repository grouping all semester pipeline work.
2. Define several CI/CD pipelines that demonstrate different DevSecOps concerns.

3. Configure GitLab CI/CD stages for quality, security, build, scan, and deployment.
4. Document the DevSecOps tools used and explain their roles.
5. Provide automation scripts for repeatable operations.
6. Include Docker and Kubernetes deployment artifacts.
7. Analyze security scan results and identify remediation actions.
8. Produce one final global report in a professional format.

1.3 Repository Organization

The final repository is organized as follows:

```
1 devsecops-project/  
2     Pipeline1/  
3     Pipeline2/  
4     Pipeline3/  
5     Pipeline4/  
6     Pipeline5/  
7     scripts/  
8     docker/  
9     kubernetes/  
10    screenshots/  
11    docs/  
12        rapport_final.tex  
13        rapport_final.html  
14        rapport_final.md  
15        rapport_final.pdf  
16    app/  
17    frontend/  
18    secops-bot/  
19    .gitlab-ci.yml  
20    docker-compose.yml  
21    README.md
```

Listing 1.1: Repository structure

Each pipeline folder contains a specific pipeline configuration and a README file explaining its purpose. The global `.gitlab-ci.yml` file combines the most important jobs into one end-to-end CI/CD workflow.

2 Application Analysis

2.1 Backend

The backend is implemented with FastAPI. It exposes two main endpoints:

- `POST /token`: authenticates a user and returns a JWT.
- `GET /secrets`: validates the JWT and returns protected demo data.

The backend demonstrates API authentication and protected resource access. From a security perspective, it is useful because it contains examples of common weaknesses that can be detected and discussed during SAST and manual review.

2.2 Frontend

The frontend is implemented with React and Vite. It provides:

- A login screen.
- JWT storage in browser local storage.
- A protected dashboard that fetches data from the backend.
- A logout action that removes the token.

The frontend is served through Nginx in the production Docker image. It is used as the DAST target for OWASP ZAP baseline scanning.

2.3 SecOps Bot

The SecOps bot is a Python service using Telegram, Elasticsearch, GitLab, and an LLM API. It provides interactive and proactive operations:

- `/siem`: queries Elasticsearch logs and summarizes suspicious events.

- `/pipelines`: queries GitLab for failed pipelines and summarizes failure traces.
- Background monitoring: searches for repeated unauthorized access events.

Sensitive values such as tokens and API keys are expected through environment variables. This is a better practice than hardcoding secrets, but the environment file must never be committed to a public repository.

2.4 Monitoring Stack

The monitoring stack uses:

- Filebeat to collect Docker container logs.
- Elasticsearch to index logs.
- Kibana to visualize and investigate events.

This monitoring layer supports the operational side of DevSecOps. It helps connect runtime events with automated incident analysis.

3 DevSecOps Architecture

3.1 Architecture Overview

The project architecture connects application services, CI/CD automation, security scanners, deployment platforms, and monitoring tools.

Table 3.1: Architecture components

Layer	Component	Role
Application	FastAPI backend	Provides authentication and protected API endpoints.
Application	React frontend	Provides user interface and communicates with the backend.
Automation	SecOps bot	Queries SIEM data and GitLab pipeline failures.
CI/CD	GitLab CI/CD	Orchestrates build, test, scan, push, DAST, and deployment stages.
SAST	SonarScanner	Detects code quality and security issues.
Container security	Trivy	Scans Docker images for high and critical vulnerabilities.
DAST	OWASP ZAP	Tests the running web application externally.
Monitoring	Filebeat, Elasticsearch, Kibana	Collects, stores, and visualizes runtime logs.
Deployment	Docker Compose	Runs the complete stack locally or in staging.
Deployment	Kubernetes	Provides a cluster deployment model.

3.2 Logical Flow

The DevSecOps flow follows these steps:

1. Developers commit code to the repository.
2. GitLab CI/CD starts the pipeline.
3. Dependencies are installed in clean runner environments.
4. Code quality and tests are executed.
5. Static analysis checks the source code.
6. Docker images are built for application components.
7. Trivy scans the generated images.
8. Images can be pushed to a registry on the main branch.
9. OWASP ZAP scans the deployed frontend.
10. Docker Compose or Kubernetes deploys the stack.
11. Filebeat forwards runtime logs to Elasticsearch.
12. Kibana and the SecOps bot support monitoring and incident analysis.

3.3 Security Philosophy

The project follows a shift-left and shield-right approach:

- Shift-left: detect security issues early during code, test, and build stages.
- Shield-right: observe the running application and respond to runtime events.

This is important because not all vulnerabilities can be found statically. Some problems appear only in the running application, while others come from infrastructure, dependencies, or user behavior.

4 CI/CD Pipeline Design

4.1 Global Pipeline

The global GitLab CI/CD file contains the following stages:

```
1 stages:
2   - install
3   - lint
4   - test
5   - sast
6   - build
7   - container_scan
8   - push
9   - dast
10  - deploy
```

Listing 4.1: Global pipeline stages

This order is intentional. Cheaper and faster checks run first, while image scanning, DAST, and deployment run later. This reduces wasted build time when source code quality or test checks already fail.

4.2 Pipeline 1: CI Quality and Unit Tests

4.2.1 Purpose

Pipeline 1 validates the codebase before security scanning and deployment. Its goal is to ensure that the application can install dependencies, pass linting, and execute tests.

4.2.2 Stages

Table 4.1: Pipeline 1 stages

Stage	Description
Install	Installs backend dependencies from <code>app/requirements.txt</code> .
Lint backend	Runs <code>flake8</code> on the FastAPI backend.
Lint frontend	Runs <code>npm run lint</code> on the React frontend.
Test	Runs <code>pytest</code> and exports coverage in Cobertura XML format.

4.2.3 Expected Evidence

The expected evidence is a GitLab pipeline graph showing successful or allowed-failure jobs, a coverage artifact, and lint logs. During a classroom demonstration, lint and test jobs may be configured with `allow_failure: true` to keep the complete pipeline visible even when the code still needs improvements.

4.3 Pipeline 2: SAST and Code Quality

4.3.1 Purpose

Pipeline 2 performs static application security testing. It scans source code for vulnerabilities, risky coding patterns, bugs, and maintainability issues.

4.3.2 Tools

The main tool is SonarScanner. A simple secret review job is also documented to search for suspicious terms such as `SECRET_KEY`, `TOKEN`, `API_KEY`, and `password`.

4.3.3 Important Findings

The source analysis identified several important findings:

- The backend contains a hardcoded JWT signing key.
- The backend enables permissive CORS using all origins.
- The backend contains demo credentials.

- The frontend contains a hardcoded API base URL.
- The SecOps bot depends on external tokens and must use environment variables safely.

4.4 Pipeline 3: Docker Build and Container Scan

4.4.1 Purpose

Pipeline 3 packages application components into Docker images and scans those images before they are pushed or deployed.

4.4.2 Images Built

- `zero-trust-vault/backend`: FastAPI backend.
- `zero-trust-vault/frontend`: React application served by Nginx.
- `zero-trust-vault/secops-bot`: Python SecOps automation bot.

4.4.3 Container Scan

Trivy scans the image artifacts for high and critical vulnerabilities. The scan checks operating system packages and application dependencies. This is necessary because a source code scan alone does not validate base images or installed packages.

4.5 Pipeline 4: DAST and Runtime Monitoring

4.5.1 Purpose

Pipeline 4 validates the running application externally and connects runtime logs to the monitoring stack.

4.5.2 DAST

OWASP ZAP baseline scanning targets the frontend URL. It performs passive security checks and produces an HTML report. Typical findings may include missing security headers, weak cookie policies, exposed routes, or other web hardening issues.

4.5.3 Runtime Monitoring

Filebeat collects Docker logs and sends them to Elasticsearch. Kibana allows analysts to search for events such as failed authentication attempts and application errors. The SecOps bot can query these logs and summarize suspicious activity.

4.6 Pipeline 5: Deployment and Automation

4.6.1 Purpose

Pipeline 5 focuses on delivery and operations. It shows how the stack can run with Docker Compose and how it can be deployed to Kubernetes.

4.6.2 Deployment Options

- Docker Compose for local and staging demonstration.
- Kubernetes manifests for cluster deployment.
- Manual deployment gates for cluster delivery.

4.6.3 Automation

The SecOps bot extends the pipeline work by connecting CI/CD failures and SIEM events to chat-based operational workflows.

5 GitLab CI/CD Configuration

5.1 Important Variables

The global pipeline defines image names and scanner target variables:

```
1 variables:
2   DOCKER_DRIVER: overlay2
3   DOCKER_TLS_CERTDIR: ""
4   BACKEND_IMAGE: "$CI_REGISTRY_IMAGE/backend:$CI_COMMIT_SHA"
5   FRONTEND_IMAGE: "$CI_REGISTRY_IMAGE/frontend:$CI_COMMIT_SHA"
6   BOT_IMAGE: "$CI_REGISTRY_IMAGE/secops-bot:$CI_COMMIT_SHA"
7   ZAP_TARGET_URL: "http://frontend"
```

Listing 5.1: Pipeline variables

Using commit SHA tags improves traceability. Each image can be linked back to the exact commit that produced it.

5.2 Artifacts

The pipeline stores important outputs as artifacts:

- `coverage.xml`: backend coverage report.
- `backend-image.tar`: backend Docker image artifact.
- `frontend-image.tar`: frontend Docker image artifact.
- `secops-bot-image.tar`: bot Docker image artifact.
- `trivy-backend.txt`: backend image scan result.
- `trivy-frontend.txt`: frontend image scan result.
- `trivy-secops-bot.txt`: bot image scan result.
- `dast_report.html`: OWASP ZAP report.

5.3 Allowed Failures

Some jobs are configured with `allow_failure: true`. In a teaching project, this makes sense because it lets the complete pipeline execute even when one quality or security check reports findings. In a production environment, this setting should be removed for critical gates such as high severity vulnerabilities, test failures, and DAST alerts.

6 DevSecOps Tools Used

Table 6.1: Tools used in the project

Tool	Category	Usage
GitLab CI/CD	Pipeline orchestration	Executes all CI/CD stages and stores artifacts.
Docker	Containerization	Packages the backend, frontend, and SecOps bot.
Docker Compose	Local deployment	Runs the full stack on one machine.
Kubernetes	Cluster deployment	Provides deployment and service manifests.
Flake8	Linting	Checks Python code style and formatting.
Pytest	Testing	Runs backend tests and generates coverage.
SonarScanner	SAST	Performs static application security and quality analysis.
Trivy	Container scanning	Detects vulnerabilities in Docker images.
OWASP ZAP	DAST	Scans the running web application.
Filebeat	Log shipping	Collects Docker container logs.
Elasticsearch	Log indexing	Stores runtime logs and makes them searchable.
Kibana	Visualization	Provides dashboards and log exploration.
Telegram Bot	Alerting interface	Allows interactive SecOps commands.
OpenRouter LLM API	/ Analysis assistance	Summarizes failed jobs and security events.

7 Security Scan Analysis

7.1 SAST Analysis

7.1.1 Hardcoded JWT Secret

The backend contains a JWT secret directly in source code:

```
1 SECRET_KEY = "SuperSecureDevSecOpsVaultKey"
```

Listing 7.1: Example of hardcoded secret pattern

Risk: If the repository becomes public or is leaked, an attacker can use the secret to forge valid tokens.

Recommendation: Move the secret to an environment variable, GitLab CI/CD protected variable, Docker secret, Kubernetes secret, or a dedicated secret manager.

7.1.2 Permissive CORS

The backend allows all origins:

```
1 allow_origins=["*"]
```

Listing 7.2: Permissive CORS pattern

Risk: Any website can call the API from a browser if authentication conditions are met. This increases exposure.

Recommendation: Restrict CORS to the known frontend domain in staging and production.

7.1.3 Demo Credentials

The authentication endpoint contains fixed demonstration credentials.

Risk: Hardcoded credentials are dangerous if deployed outside a lab environment.

Recommendation: Replace the demo logic with a real identity provider or a database-backed user store with hashed passwords.

7.1.4 Hardcoded Frontend API URL

The frontend contains a fixed backend URL.

Risk: The build becomes tied to one environment and may expose internal infrastructure details.

Recommendation: Use environment-specific variables such as `VITE_API_BASE_URL`.

7.2 Container Scan Analysis

Trivy is configured to scan backend, frontend, and SecOps bot images for high and critical vulnerabilities.

7.2.1 Expected Remediation Process

1. Review the Trivy report artifact.
2. Identify whether vulnerabilities come from base images or application dependencies.
3. Update base images such as `python:3.11-slim`, `node:20-alpine`, and `nginx:stable-alpine`.
4. Update Python and Node.js dependencies.
5. Rebuild images and rescan.
6. Document accepted risks when no fix is available.

7.3 DAST Analysis

OWASP ZAP baseline scanning checks the web application from the outside. This is complementary to SAST and container scanning because it tests the real HTTP behavior of the deployed application.

7.3.1 Typical Findings

Possible ZAP findings include:

- Missing `Content-Security-Policy` header.
- Missing `X-Frame-Options` or equivalent frame protection.
- Missing `X-Content-Type-Options` header.
- Missing secure cookie attributes.
- Information disclosure through headers.

7.3.2 Recommended Hardening

- Add strict security headers at the Nginx or backend layer.
- Use HTTPS in production.
- Restrict exposed routes.
- Avoid storing sensitive JWTs in local storage for high-risk applications.

7.4 Runtime Monitoring Analysis

The monitoring stack collects and analyzes logs after deployment. The SecOps bot watches for repeated unauthorized access patterns and errors.

7.4.1 Observed Event Types

- HTTP 401 Unauthorized responses.
- Backend application errors.
- Container startup and restart events.
- Pipeline failures from GitLab.

7.4.2 Operational Value

This layer helps detect problems that scanners cannot fully predict. For example, repeated failed authentication attempts can indicate brute force activity or misuse of credentials.

8 Docker Deployment

8.1 Docker Compose Services

The Docker Compose file defines these services:

Table 8.1: Docker Compose services

Service	Image / Build	Purpose
backend	<code>docker/backend.Dockerfile</code>	Runs the FastAPI API on port 8000.
frontend	<code>docker/frontend.Dockerfile</code>	Serves the React app through Nginx on port 5173.
elasticsearch	Elastic official image	Stores and indexes logs.
kibana	Elastic official image	Visualizes logs and dashboards.
filebeat	Elastic official image	Reads Docker logs and forwards them to Elasticsearch.
secops-bot	<code>docker/secops-bot.Dockerfile</code>	Runs the Telegram/GitLab/SIEM automation bot.

8.2 Execution

```
1 docker compose up -d
```

Listing 8.1: Run the stack with Docker Compose

8.3 Security Notes

The Docker Compose deployment is suitable for local demonstration. For production, the project should enable TLS, authenticated Elasticsearch, secret storage, network restrictions, and stronger health checks.

9 Kubernetes Deployment

9.1 Manifests

The Kubernetes folder contains:

- namespace.yaml
- backend-deployment.yaml
- frontend-deployment.yaml
- elk-stack.yaml
- secops-bot-deployment.yaml
- kustomization.yaml

9.2 Deployment Command

```
1 kubectl apply -k kubernetes/
```

Listing 9.1: Deploy with Kustomize

9.3 Kubernetes Security Improvements

The provided manifests are sufficient for demonstration, but a production cluster should also include:

- Resource requests and limits.
- Network policies.
- Read-only root filesystems where possible.
- Non-root containers.

- External secret management.
- Ingress with TLS.
- Pod security standards.

10 Automation Scripts

10.1 Script Inventory

Table 10.1: Automation scripts

Script	Purpose
<code>build-images.sh</code>	Builds backend, frontend, and SecOps bot Docker images.
<code>security-scan.sh</code>	Runs Trivy scans and writes reports.
<code>run-dast.sh</code>	Runs OWASP ZAP baseline scan against a target URL.
<code>deploy-k8s.sh</code>	Applies the Kubernetes manifests.
<code>collect-evidence.sh</code>	Collects service status files and refreshes screenshot checklist.

10.2 Automation Benefits

The scripts make the project repeatable. A user can build images, run scans, deploy manifests, and collect evidence without manually retyping long commands.

11 Risk Register

Table 11.1: Project risk register

Risk	Description	Severity	Mitigation
Hardcoded JWT secret	Secret is stored directly in source code.	High	Move to protected environment variables or secret manager.
Permissive CORS	API accepts requests from all origins.	Medium	Restrict allowed origins per environment.
Demo credentials	Fixed login credentials exist in code.	High	Implement secure authentication and password hashing.
Hardcoded API URL	Frontend target is fixed in source.	Medium	Use environment-based configuration.
Unprotected Elasticsearch	Security disabled for demonstration.	High	Enable authentication and TLS in production.
Allowed failure jobs	Security findings may not block deployment.	Medium	Enforce quality gates before production release.
Missing screenshots	Evidence may be incomplete before submission.	Low	Add screenshots listed in the evidence folder.

12 Recommendations

12.1 Short-Term Improvements

- Replace hardcoded secrets with environment variables.
- Add a `.env.example` file and keep real secrets out of Git.
- Restrict CORS to the frontend domain.
- Configure frontend API URL through `VITE_API_BASE_URL`.
- Add basic backend tests for authentication success, authentication failure, and protected endpoint access.
- Add Nginx security headers for the frontend container.

12.2 Medium-Term Improvements

- Enforce SonarQube quality gates.
- Fail the pipeline on high and critical vulnerabilities when fixes are available.
- Add dependency scanning for Python and Node.js packages.
- Add GitLab protected environments and manual approval before production deployment.
- Add Kubernetes network policies and resource limits.

12.3 Long-Term Improvements

- Integrate a real identity provider.
- Use a secret manager such as HashiCorp Vault or cloud-native secrets.
- Add centralized alerting for SIEM events.

- Add infrastructure-as-code scanning.
- Add software bill of materials generation.
- Add signed container images and admission control.

13 Conclusion

This project demonstrates a complete DevSecOps workflow across the software delivery lifecycle. It starts with CI quality checks, continues with SAST and container scanning, validates the running application with DAST, and supports deployment through Docker Compose and Kubernetes. The monitoring stack and SecOps bot extend the work beyond deployment by adding runtime visibility and automated analysis.

The project also shows that DevSecOps is not only about adding tools. The most important value comes from connecting tools into a coherent workflow, documenting evidence, and using scan results to drive remediation. The current project is appropriate for a semester demonstration, and it provides a strong base for future production hardening.

A Useful Commands

```
1 ./scripts/build-images.sh
```

Listing A.1: Build Docker images

```
1 ./scripts/security-scan.sh
```

Listing A.2: Run container vulnerability scans

```
1 ./scripts/run-dast.sh http://localhost:5173
```

Listing A.3: Run DAST scan

```
1 docker compose up -d
```

Listing A.4: Deploy locally

```
1 ./scripts/deploy-k8s.sh
```

Listing A.5: Deploy to Kubernetes

B Compilation

To compile this LaTeX report on a machine with a LaTeX distribution installed:

```
1 cd devsecops-project/docs
2 pdflatex rapport_final.tex
3 pdflatex rapport_final.tex
```

Listing B.1: Compile the LaTeX report

Running the command twice updates the table of contents, list of figures, and list of tables.